第4章 控制执行流程

就像有知觉的生物一样，程序必须在执行过程中控制它的世界，并做出选择。在Java中，你要使用执行控制语句来做出选择。

Java使用了C的所有流程控制语句，所以如果读者以前用过C或C++ 编程，那么应该非常熟悉了。大多数过程型编程语言都具有某些形式的控制语句，它们通常在各种语言间是交迭的。在Java中，涉及的关键字包括**if-else**、**while**、**do-while**、**for**、**return**、**break**以及选择语句**switch**。然而，Java并不支持**goto**语句（该语句引起许多反对意见，但它仍是解决某些特殊问题的最便利的方法）。在Java中，仍然可以进行类似**goto**那样的跳转，但比起典型的**goto**，有了很多限制。

4.1 true和false

所有条件语句都利用条件表达式的真或假来决定执行路径。这里有一个条件表达式的例子：**a==b**。它用条件操作符“==”来判断**a**值是否等于**b**值。该表达式返回**true**或**false**。本章前面介绍的所有关系操作符，都可拿来构造条件语句。注意Java不允许我们将一个数字作为布尔值使用，虽然这在C和C++里是允许的（在这些语言里，“真”是非零，而“假”是零）。如果想在布尔测试中使用一个非布尔值，比如在**if(a)**中，那么首先必须用一个条件表达式将其转换成布尔值，例如**if(a!=0)**。

4.2 if-else

if-else语句是控制程序流程的最基本的形式。其中的**else**是可选的，所以可按下述两种形式来使用**if**：

135

```
if(Boolean-expression)
  statement
```

或

```
if(Boolean-expression)
  statement
else
  statement
```

布尔表达式必须产生一个布尔结果，statement指用分号结尾的简单语句，或复合语句——封闭在花括号内的一组简单语句。在本书任何地方，只要提及“语句”这个词，就指的是简单语句或复合语句。

作为**if-else**的一个例子，下面这个**test()**方法可以告诉您，您猜的数是大于、小于还是等于目标数：

```
//: control/IfElse.java
import static net.mindview.util.Print.*;

public class IfElse {
  static int result = 0;
  static void test(int testval, int target) {
    if(testval > target)
      result = +1;
    else if(testval < target)
```

```
      result = -1;
    else
      result = 0; // Match
  }
  public static void main(String[] args) {
    test(10, 5);
    print(result);
    test(5, 10);
    print(result);
    test(5, 5);
    print(result);
  }
} /* Output:
1
-1
0
*///:~
```

136

在test()的中间部分，可以看到一个“**else if**”，那并非新的关键字，而仅仅只是一个**else**后面紧跟另一个新的**if**语句。

尽管Java与它之前产生的C和C++一样，都是“格式自由”的语言，但是习惯上还是将流程控制语句的主体部分缩进排列，使读者能方便地确定起始与终止。

4.3 迭代

while、**do-while**和**for**用来控制循环，有时将它们划分为*迭代语句*（iteration statement）。语句会重复执行，直到起控制作用的*布尔表达式*（Booleanexpression）得到“假”的结果为止。**while**循环的格式如下：

```
while(Boolean-expression)
  statement
```

在循环刚开始时，会计算一次布尔表达式的值；而在语句的下一次迭代开始前会再计算一次。

下面这个简单的例子可产生随机数，直到符合特定的条件为止：

```
//: control/WhileTest.java
// Demonstrates the while loop.

public class WhileTest {
  static boolean condition() {
    boolean result = Math.random() < 0.99;
    System.out.print(result + ", ");
    return result;
  }
  public static void main(String[] args) {
    while(condition())
      System.out.println("Inside 'while'");
    System.out.println("Exited 'while'");
  }
} /* (Execute to see output) *///:~
```

condition()方法用到了**Math**库里的**static**（静态）方法**random()**，该方法的作用是产生0和1之间（包括0，但不包括1）的一个**double**值。**result**的值是通过比较操作符<而得到它，这个操作符将产生**boolean**类型的结果。在打印**boolean**类型的值时，将自动地得到适合的字符串true或false。**while**的条件表达式意思是说：“只要**condition()**返回**true**，就重复执行循环体中的语句”。 137

4.3.1 do-while

do-while的格式如下：

```
do
  statement
while(Boolean-expression);
```

while和**do-while**唯一的区别就是**do-while**中的语句至少会执行一次，即便表达式第一次就被计算为**false**。而在**while**循环结构中，如果条件第一次就为**false**，那么其中的语句根本不会执行。在实际应用中，**while**比**do-while**更常用一些。

4.3.2 for

for循环可能是最经常使用的迭代形式，这种在第一次迭代之前要进行初始化。随后，它会进行条件测试，而且在每一次迭代结束时，进行某种形式的“步进”。**for**循环的格式如下：

```
for(initialization; Boolean-expression; step)
  statement
```

初始化（initialization）表达式、*布尔表达式*（Boolean-expression），或者*步进*（step）运算，都可以为空。每次迭代前会测试布尔表达式。若获得的结果是**false**，就会执行**for**语句后面的代码行。每次循环结束，会执行一次步进。

for循环常用于执行“计数”任务：

```
//: control/ListCharacters.java
// Demonstrates "for" loop by listing
// all the lowercase ASCII letters.

public class ListCharacters {
  public static void main(String[] args) {
    for(char c = 0; c < 128; c++)
      if(Character.isLowerCase(c))
        System.out.println("value: " + (int)c +
          " character: " + c);
  }
} /* Output:
value: 97 character: a
value: 98 character: b
value: 99 character: c
value: 100 character: d
value: 101 character: e
value: 102 character: f
value: 103 character: g
value: 104 character: h
value: 105 character: i
value: 106 character: j
...
*///:~
```

138

注意，变量**c**是在程序用到它的地方被定义的，也就是在for循环的控制表达式里，而不是在**main()**开始的地方定义的。**c**的作用域就是**for**控制的表达式的范围内。

这个程序也使用了**java.lang.Character**包装器类，这个类不但能把**char**基本类型的值包装进对象，还提供了一些别的有用的方法。这里用到了**static isLowerCase()**方法来检查问题中的字符是否为小写字母。

对于像C语言那样的传统的过程型语言，要求所有变量都在一个块的开头定义，以便编译器在创建这个块的时候，可以为那些变量分配空间。而在Java和C++中，则可在整个块的范围内分散变量声明，在真正需要的地方才加以定义。这样便可形成更自然的编程风格，也更易理解。

练习1：(1) 写一个程序，打印从1到100的值。

练习2：(2) 写一个程序，产生25个**int**类型的随机数。对于每一个随机值，使用**if-else**语句来将其分类为大于、小于，或等于紧随它而随机生成的值。

练习3：(1) 修改练习2，把代码用一个**while**无限循环包括起来。然后运行它直至用键盘中断其运行（通常是通过按Ctrl-C）。

练习4：(3) 写一个程序，使用两个嵌套的**for**循环和取余操作符（%）来探测和打印素数（只能被其自身和1整除，而不能被其他数字整除的整数）。

139

练习5：(4) 重复第3章中的练习10，不要用**Integer.toBinaryString()**方法，而是用三元操作符和按位操作符来显示二进制的1和0。

4.3.3 逗号操作符

本章前面已经提到了逗号操作符（注意不是逗号分隔符，逗号用作分隔符时用来分隔函数的不同参数），Java里唯一用到逗号操作符的地方就是**for**循环的控制表达式。在控制表达式的初始化和步进控制部分，可以使用一系列由逗号分隔的语句；而且那些语句均会独立执行。

通过使用逗号操作符，可以在**for**语句内定义多个变量，但是它们必须具有相同的类型。

```
//: control/CommaOperator.java

public class CommaOperator {
  public static void main(String[] args) {
    for(int i = 1, j = i + 10; i < 5; i++, j = i * 2) {
      System.out.println("i = " + i + " j = " + j);
    }
  }
} /* Output:
i = 1 j = 11
i = 2 j = 4
i = 3 j = 6
i = 4 j = 8
*///:~
```

for语句中的**int**定义涵盖了**i**和**j**，在初始化部分实际上可以拥有任意数量的具有相同类型的变量定义。在一个控制表达式中，定义多个变量的这种能力只限于**for**循环适用，在其他任何选择或迭代语句中都不能使用这种方式。

可以看到，无论在初始化还是在步进部分，语句都是顺序执行的。此外，初始化部分可以有任意数量的同一类型的定义。

4.4 Foreach语法

Java SE5引入了一种新的更加简洁的**for**语法用于数组和容器（在第16章与第17章中将更多地讨论这种语法），即foreach语法，表示不必创建**int**变量去对由访问项构成的序列进行计数，foreach将自动产生每一项。

140

例如，假设有一个**float**数组，我们要选取该数组中的每一个元素：

```
//: control/ForEachFloat.java
import java.util.*;

public class ForEachFloat {
  public static void main(String[] args) {
    Random rand = new Random(47);
    float f[] = new float[10];
    for(int i = 0; i < 10; i++)
      f[i] = rand.nextFloat();
    for(float x : f)
      System.out.println(x);
  }
} /* Output:
0.72711575
0.39982635
```

```
0.5309454
0.0534122
0.16020656
0.57799757
0.18847865
0.4170137
0.51660204
0.73734957
*///:~
```

这个数组是用旧式的**for**循环组装的，因为在组装时必须按索引访问它。在下面这行中可以看到foreach语法：

```
for(float x : f) {
```

这条语句定义了一个**float**类型的变量**x**，继而将每一个**f**的元素赋值给**x**。

任何返回一个数组的方法都可以使用foreach。例如，**String**类有一个方法**toCharArray()**，它返回一个**char**数组，因此可以很容易地像下面这样迭代在字符串里面的所有字符：

141

```
//: control/ForEachString.java
public class ForEachString {
  public static void main(String[] args) {
    for(char c : "An African Swallow".toCharArray() )
      System.out.print(c + " ");
  }
} /* Output:
A n  A f r i c a n  S w a l l o w
*///:~
```

就像在第11章中所看到的，foreach还可以用于任何**Iterable**对象。

许多**for**语句都会在一个整型值序列中步进，就像下面这样：

```
for(int i = 0; i < 100; i++)
```

对于这些语句，foreach语法将不起作用，除非先创建一个**int**数组。为了简化这些任务，我在**net.mindview.util.Range**包中创建了一个名为**range()**的方法，它可以自动地生成恰当的数组。我的目的是将**range()**用做**static**导入：

```
//: control/ForEachInt.java
import static net.mindview.util.Range.*;
import static net.mindview.util.Print.*;

public class ForEachInt {
  public static void main(String[] args) {
    for(int i : range(10)) // 0..9
      printnb(i + " ");
    print();
    for(int i : range(5, 10)) // 5..9
      printnb(i + " ");
    print();
    for(int i : range(5, 20, 3)) // 5..20 step 3
      printnb(i + " ");
    print();
  }
} /* Output:
0 1 2 3 4 5 6 7 8 9
5 6 7 8 9
5 8 11 14 17
*///:~
```

142

range()方法已经被重载，重载表示相同的方法名可以具有不同的参数列表（你将很快学习重载）。**range()**的第一种重载形式是从0开始产生值，直至范围的上限，但不包括该上限。第二种形式从第一个值开始产生值，直至比第二个值小1的值为止。第三种形式有一个步进值，因此

它每次的增量为该值。**range()**是所谓生成器的一个非常简单的版本，有关生成器的内容将在本书稍后进行介绍。

请注意，尽管**range()**使得foreach语法可以适用于更多的场合，并且这样做似乎可以增加可读性，但是它的效率会稍许降低，因此如果您在做性能调优，也许应该使用仿真器来做评价，它是一种可以度量代码性能的工具。

你会注意到，除了**print()**之外，我们还使用了**printnb()**。**printnb()**方法不会换行，因此可以使用它将一行拆分成多个片断输出。

foreach语法不仅在录入代码时可以节省时间，更重要的是，它阅读起来也要容易得多，它说明您正在努力做什么（例如获取数组中的每一个元素），而不是给出你正在如何做的细节（例如正在创建索引，因此可以使用它来选取数组中的每一个元素）。在本书中，我们只要有可能就会使用foreach语法。

4.5 return

在Java中有多个关键词表示无条件分支，它们只是表示这个分支无需任何测试即可发生。这些关键词包括**return**、**break**、**continue**和一种与其他语言中的goto类似的跳转到标号语句的方式。

return关键词有两方面的用途：一方面指定一个方法返回什么值（假设它没有**void**返回值），另一方面它会导致当前的方法退出，并返回那个值。可据此改写上面的**test()**方法，使其利用这些特点：

```
//: control/IfElse2.java
import static net.mindview.util.Print.*;

public class IfElse2 {
  static int test(int testval, int target) {
    if(testval > target)
      return +1;
    else if(testval < target)
      return -1;
    else
      return 0; // Match
  }
  public static void main(String[] args) {
    print(test(10, 5));
    print(test(5, 10));
    print(test(5, 5));
  }
} /* Output:
1
-1
0
*///:~
```

143

不必加上**else**，因为方法在执行了**return**后不再继续执行。

如果在返回**void**的方法中没有**return**语句，那么在该方法的结尾处会有一个隐式的**return**，因此在方法中并非总是必须要有一个**return**语句。但是，如果一个方法声明它将返回void之外的其他东西，那么必须确保每一条代码路径都将返回一个值。

练习6：(2) 修改前两个程序中的两个**test()**方法，让它们接受两个额外的参数**begin**和**end**，这样在测试**testval**时将判断它是否在**begin**和**end**之间（包括**begin**和**end**）的范围内。

4.6 break和 continue

在任何迭代语句的主体部分，都可用**break**和**continue**控制循环的流程。其中，**break**用于强

行退出循环，不执行循环中剩余的语句。而**continue**则停止执行当前的迭代，然后退回循环起始处，开始下一次迭代。

下面这个程序向大家展示了**break**和**continue**在**for**和**while**循环中的例子：

144

```
//: control/BreakAndContinue.java
// Demonstrates break and continue keywords.
import static net.mindview.util.Range.*;
public class BreakAndContinue {
  public static void main(String[] args) {
    for(int i = 0; i < 100; i++) {
      if(i == 74) break; // Out of for loop
      if(i % 9 != 0) continue; // Next iteration
      System.out.print(i + " ");
    }
    System.out.println();
    // Using foreach:
    for(int i : range(100)) {
      if(i == 74) break; // Out of for loop
      if(i % 9 != 0) continue; // Next iteration
      System.out.print(i + " ");
    }
    System.out.println();
    int i = 0;
    // An "infinite loop":
    while(true) {
      i++;
      int j = i * 27;
      if(j == 1269) break; // Out of loop
      if(i % 10 != 0) continue; // Top of loop
      System.out.print(i + " ");
    }
  }
} /* Output:
0 9 18 27 36 45 54 63 72
0 9 18 27 36 45 54 63 72
10 20 30 40
*///:~
```

在这个**for**循环中，**i**的值永远不会达到100；因为一旦**i**到达74，**break**语句就会中断循环。通常，只有在不知道中断条件何时满足时，才需要这样使用**break**。只要**i**不能被9整除，**continue**语句就会使执行过程返回到循环的最开头（这使i值递增）。如果能够整除，则将值显示出来。

第二种for循环展示了foreach用法，它将产生相同的结果。

145

最后，可以看到一个“无穷**while**循环”的情况。然而，循环内部有一个**break**语句，可中止循环。除此以外，大家还会看到**continue**语句执行序列移回到循环的开头，而没有去完成**continue**语句之后的所有内容。（只有在**i**值能被10整除时才打印出值。）输出结果之所以显示0，是由于0%9等于0。

无穷循环的第二种形式是**for(;;)**。编译器将**while(true)**与**for(;;)**看作是同一回事。所以具体选用哪个取决于自己的编程习惯。

练习7：(1) 修改本章练习1，通过使用**break**关键词，使得程序在打印到99时退出。然后尝试使用**return**来达到相同的目的。

4.7 臭名昭著的goto

编程语言中一开始就有**goto**关键词了。事实上，**goto**起源于汇编语言的程序控制：“若条件A成立，则跳到这里；否则跳到那里”。如果阅读由编译器最终生成的汇编代码，就会发现程序

控制里包含了许多跳转。(Java编译器生成它自己的“汇编代码”，但是这个代码是运行在Java虚拟机上的，而不是直接运行在CPU硬件上。)

goto语句是在源码级上的跳转，这使其招致了不好的声誉。若一个程序总是从一个地方跳到另一个地方，还有什么办法能识别程序的控制流程呢？自从Edsger Dijkstra发表了著名论文《Goto considered harmful》(Goto有害)，众人开始痛斥**goto**的不是，甚至建议将它从关键词集合中扫地出门。

对于这个问题，中庸之道是最好解决方法。真正的问题并不在于使用**goto**，而在于**goto**的滥用；而且少数情况下，**goto**还是组织控制流程的最佳手段。

尽管**goto**仍是Java中的一个保留字，但在语言中并未使用它；Java没有**goto**。然而，Java也能完成一些类似于跳转的操作，这与**break**和**continue**这两个关键词有关。它们其实不是一个跳转，而是中断迭代语句的一种方法。之所以把它们纳入**goto**问题中一起讨论，是由于它们使用了相同的机制：标签。

标签是后面跟有冒号的标识符，就像下面这样：

146

```
label1:
```

在Java中，标签起作用的唯一的地方刚好是在迭代语句之前。“刚好之前”的意思表明，在标签和迭代之间置入任何语句都不好。而在迭代之前设置标签的唯一理由是：我们希望在其中嵌套另一个迭代或者一个开关（你很快就会学习到它）。这是由于**break**和**continue**关键词通常只中断当前循环，但若随同标签一起使用，它们就会中断循环，直到标签所在的地方：

```
label1:
outer-iteration {
  inner-iteration {
    //...
    break; // (1)
    //...
    continue;  // (2)
    //...
    continue label1; // (3)
    //...
    break label1;  // (4)
  }
}
```

在**(1)**中，**break**中断内部迭代，回到外部迭代。在**(2)**中，**continue**使执行点移回内部迭代的起始处。在**(3)**中，**continue label1**同时中断内部迭代以及外部迭代，直接转到**label1**处；随后，它实际上是继续迭代过程，但却从外部迭代开始。在**(4)**中，**break label1**也会中断所有迭代，并回到**label1**处，但并不重新进入迭代。也就是说，它实际是完全中止了两个迭代。

下面是标签用于**for**循环的例子：

```
//: control/LabeledFor.java
// For loops with "labeled break" and "labeled continue."
import static net.mindview.util.Print.*;

public class LabeledFor {
  public static void main(String[] args) {
    int i = 0;
    outer: // Can't have statements here
    for(; true ;) { // infinite loop
      inner: // Can't have statements here
      for(; i < 10; i++) {
        print("i = " + i);
        if(i == 2) {
          print("continue");
```

147

```
          continue;
        }
        if(i == 3) {
          print("break");
          i++; // Otherwise i never
               // gets incremented.
          break;
        }
        if(i == 7) {
          print("continue outer");
          i++; // Otherwise i never
               // gets incremented.
          continue outer;
        }
        if(i == 8) {
          print("break outer");
          break outer;
        }
        for(int k = 0; k < 5; k++) {
          if(k == 3) {
            print("continue inner");
            continue inner;
          }
        }
      }
    }
    // Can't break or continue to labels here
  }
} /* Output:
i = 0
continue inner
i = 1
continue inner
i = 2
continue
i = 3
break
i = 4
continue inner
i = 5
continue inner
i = 6
continue inner
i = 7
continue outer
i = 8
break outer
*///:~
```

148

注意，**break**会中断**for**循环，而且在抵达**for**循环的末尾之前，递增表达式不会执行。由于**break**跳过了递增表达式，所以在**i==3**的情况下直接对i执行递增运算。在**i==7**的情况下，**continue outer**语句会跳到循环顶部，而且也会跳过递增，所以这里也对**i**直接递增。

如果没有**break outer**语句，就没有办法从内部循环里跳出外部循环。这是由于**break**本身只能中断最内层的循环（**continue**同样也是如此）。

当然，如果想在中断循环的同时退出，简单地用一个**return**即可。

下面这个例子向大家展示了带标签的**break**以及**continue**语句在**while**循环中的用法：

```
//: control/LabeledWhile.java
// While loops with "labeled break" and "labeled continue."
import static net.mindview.util.Print.*;

public class LabeledWhile {
  public static void main(String[] args) {
    int i = 0;
```

```
    outer:
    while(true) {
      print("Outer while loop");
      while(true) {
        i++;
        print("i = " + i);
        if(i == 1) {
          print("continue");
          continue;
        }
        if(i == 3) {
          print("continue outer");
          continue outer;
        }
        if(i == 5) {
          print("break");
          break;
        }
        if(i == 7) {
          print("break outer");
          break outer;
        }
      }
    }
  }
} /* Output:
Outer while loop
i = 1
continue
i = 2
i = 3
continue outer
Outer while loop
i = 4
i = 5
break
Outer while loop
i = 6
i = 7
break outer
*///:~
```

149

同样的规则亦适用于**while**：

1) 一般的**continue**会退回最内层循环的开头（顶部），并继续执行。

2) 带标签的**continue**会到达标签的位置，并重新进入紧接在那个标签后面的循环。

3) 一般的**break**会中断并跳出当前循环。

4) 带标签的**break**会中断并跳出标签所指的循环。

150

要记住的重点是：在Java里需要使用标签的唯一理由就是因为有循环嵌套存在，而且想从多层嵌套中**break**或**continue**。

在Dijkstra的《Goto有害》的论文中，他最反对的就是标签，而非**goto**。他发现在一个程序里随着标签的增多，产生的错误也会越来越多，并且标签和**goto**使得程序难以分析。但是，Java的标签不会造成这种问题，因为它们的应用场合已经受到了限制，没有特别的方式用于改变程序的控制。由此也引出了一个有趣的现象：通过限制语句的能力，反而能使一项语言特性更加有用。

4.8 switch

switch有时也被划归为一种*选择语句*。根据整数表达式的值，**switch**语句可以从一系列代码中选出一段去执行。它的格式如下：

```
switch(integral-selector) {
  case integral-value1 : statement; break;
  case integral-value2 : statement; break;
  case integral-value3 : statement; break;
  case integral-value4 : statement; break;
  case integral-value5 : statement; break;
  // ...
  default: statement;
}
```

其中，Integral-selector（整数选择因子）是一个能够产生整数值的表达式，**switch**能将这个表达式的结果与每个integral-value（整数值）相比较。若发现相符的，就执行对应的语句（单一语句或多条语句，其中并不需要括号）。若没有发现相符的，就执行**default**（默认）语句。

在上面的定义中，大家会注意到每个**case**均以一个**break**结尾，这样可使执行流程跳转至**switch**主体的末尾。这是构建**switch**语句的一种传统方式，但**break**是可选的。若省略**break**，会继续执行后面的**case**语句，直到遇到一个**break**为止。尽管通常不想出现这种情况，但对有经验的程序员来说，也许能够善加利用这种情况。注意最后的**default**语句没有**break**，因为执行流程已到了**break**的跳转目的地。当然，如果考虑到编程风格方面的原因，完全可以在**default**语句的末尾放置一个**break**，尽管它并没有任何实际的用处。

151

switch语句是实现多路选择（也就是说从一系列执行路径中挑选一个）的一种干净利落的方法。但它要求使用一个选择因子，并且必须是**int**或**char**那样的整数值。例如，假若将一个字符串或者浮点数作为选择因子使用，那么它们在**switch**语句里是不会工作的。对于非整数类型，则必须使用一系列**if**语句。在下一章的末尾，你将看到Java SE5的新特性**enum**，它可以帮助我们减弱这种限制，因为**enum**可以和**switch**协调工作。

下面这个例子可随机生成字母，并判断它们是元音还是辅音字母：

```
//: control/VowelsAndConsonants.java
// Demonstrates the switch statement.
import java.util.*;
import static net.mindview.util.Print.*;

public class VowelsAndConsonants {
  public static void main(String[] args) {
    Random rand = new Random(47);
    for(int i = 0; i < 100; i++) {
      int c = rand.nextInt(26) + 'a';
      printnb((char)c + ", " + c + ": ");
      switch(c) {
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u': print("vowel");
                  break;
        case 'y':
        case 'w': print("Sometimes a vowel");
                  break;
        default:  print("consonant");
      }
    }
  }
} /* Output:
y, 121: Sometimes a vowel
n, 110: consonant
z, 122: consonant
b, 98: consonant
r, 114: consonant
n, 110: consonant
```

152

```
y, 121: Sometimes a vowel
g, 103: consonant
c, 99: consonant
f, 102: consonant
o, 111: vowel
w, 119: Sometimes a vowel
z, 122: consonant
...
*///:~
```

由于**Random.nextInt(26)**会产生0到26之间的一个值，所以在其上加上一个偏移量“a”，即可产生小写字母。在**case**语句中，使用单引号引起的字符也会产生用于比较的整数值。

请注意**case**语句能够堆叠在一起，为一段代码形成多重匹配，即只要符合多种条件中的一种，就执行那段特别的代码。这时也应注意将**break**语句置于特定**case**的末尾，否则控制流程会简单地下移，处理后面的**case**。

在下面的语句中：

```
int c = rand.nextInt(26) + 'a';
```

Random.nextInt()将产生0～25之间的一个随机**int**值，它将被加到“**a**”上。这表示“**a**”将自动被转换为**int**以执行加法。为了把**c**当作字符打印，必须将其转型为**char**；否则，将产生整型输出。

练习8：(2) 写一个**switch**开关语句，为每个**case**打印一个消息。然后把这个**switch**放进**for**循环来测试每个**case**。先让每个**case**后面都有**break**，测试一下会怎样；然后把break删了，看看会怎样。

练习9：(4) 一个斐波那契数列是由数字1、1、2、3、5、8、13、21、34等等组成的，其中每一个数字（从第三个数字起）都是前两个数字的和。创建一个方法，接受一个整数参数，并显示从第一个元素开始总共由该参数指定的个数所构成的所有斐波那契数字。例如，如果运行**java Fibonacci 5**（其中**Fibonacci**是类名），那么输出就应该是1、1、2、3、5。 153

练习10：(5) 吸血鬼数字是指位数为偶数的数字，可以由一对数字相乘而得到，而这对数字各包含乘积的一半位数的数字，其中从最初的数字中选取的数字可以任意排序。以两个0结尾的数字是不允许的，例如，下列数字都是“吸血鬼”数字：

1260 = 21 * 60

1827 = 21 * 87

2187 = 27 * 81

写一个程序，找出4位数的所有吸血鬼数字（Dan Forhan推荐）。

4.9 总结

本章介绍了大多数编程语言都具有的基本特性：运算、操作符优先级、类型转换以及选择和循环等等。现在，我们已经做好了准备，以使自己更靠近面向对象的程序设计的世界。在下一章里，我们将讨论对象的初始化与清理，再后面则讲述隐藏实现细节（implementation hiding）这一核心概念。

所选习题的答案都可以在名为The Thinking in Java Annotated Solution Guide的电子文档中找到，读者可以从www.MindView.net处购买此文档。 154